

Procedimento — instalar e afinar uma câmara nova para o kTAMV

23/09/2026 · Impressoras IDEX Blocks

O processo num relance

Uma câmara nova afina-se em 7 passos, por esta ordem: cada passo depende do anterior. O foco (Z) vem antes da exposição porque a melhor exposição muda com a distância; ambos vêm antes da validação porque os scripts usam o próprio detetor do kTAMV para medir.



Conta com cerca de uma hora. Se o detetor não encontrar o bico no passo 5 ou 7, segue a secção **Se o detetor falhar** antes de continuar.

Antes de começar

Os quatro scripts de afinação ficam em `/home/blocks/` e correm por SSH com `python3 ~/<script>`. Cada um tem, no topo, constantes que dependem da câmara: **edita-as antes de correr** se a câmara, a porta USB ou a posição mudaram.

Script	Para quê	Constantes a rever
<code>ktamv_find_focus.py</code>	Só para câmaras com foco motorizado: varre <code>focus_absolute</code> e escolhe o mais nítido	<code>DEVICE</code> , <code>SNAPSHOT_URL</code> , <code>FOCUS_MIN</code> / <code>FOCUS_MAX</code>
<code>ktamv_find_z.py</code>	Varre a altura do prato e mede nitidez e detecção; dá o melhor Z e a margem	<code>SNAPSHOT_URL</code> , <code>CAM_X</code> / <code>CAM_Y</code> , <code>Z_MIN</code> / <code>Z_MAX</code> / <code>Z_STEP</code>
<code>ktamv_find_exposure.py</code>	Testa várias exposições com o detetor real (deteções, jitter, saturação)	<code>DEVICE</code> , <code>SNAPSHOT_URL</code> , <code>EXPOSURES</code>
<code>ktamv_capture_frames.py</code>	Guarda imagens cruas em várias exposições em <code>~/ktamv_frames/</code> , para análise	<code>DEVICE</code> , <code>SNAPSHOT_URL</code> , <code>EXPOSURES</code> e o valor em que deixa a câmara no fim

Os scripts de Z e de exposição importam o detetor de `/home/blocks/kTAMV/server/`: medem com exatamente o mesmo algoritmo que a calibração usa.

Precisas também de acesso SSH (PowerShell: `ssh blocks@<IP>`), do WinSCP para copiar ficheiros, e de `v4l-utils` instalado (`sudo apt install v4l-utils`).

Guarda uma cópia dos quatro scripts no repositório de configs, para não dependerem de uma máquina em concreto.

Passo 1 — Ligar e identificar a câmara

A câmara do bico tem de ficar na `[cam 2]` do `crownsnest.conf` (URL `/webcam2/`), em **640x480 (4:3)**. O servidor do kTAMV redimensiona tudo para 640x480 sem manter a proporção: uma câmara em 16:9 deforma o bico e o furo deixa de parecer redondo.

- 1 Liga a câmara e lista os dispositivos: `ls /dev/v4l/by-path/`. O novo é o que aparece a mais em relação à lista antiga; usa sempre o caminho `by-path` (fixo por porta USB), nunca `/dev/videoN`.
- 2 Confirma que a câmara suporta 640x480: `v4l2-ctl -d <device> --list-formats-ext`. Prefere o formato MJPG, que não carrega o processador do Pi.
- 3 Na `[cam 2]` do `crownsnest.conf`, muda `device`: para o novo caminho e confirma `resolution: 640x480` e `max_fps: 15`.

4 Reinicia o crowsnest: `sudo systemctl restart crowsnest`.

5 Confirma no browser que `http://<IP>/webcam2/snapshot?max_delay=0` mostra a imagem do bico.

Mantém os comentários do `crowsnest.conf` alinhados com as secções (NOZZLE CAM em cima da [cam 2]), para ninguém confundir as câmaras.

Passo 2 — Controlos da câmara

O objetivo é tirar à câmara tudo o que é automático: a deteção depende de a imagem ser igual de frame para frame. Lista os controlos:

```
v4l2-ctl -d <device> --list-ctrls
```

Procura estes controlos (os nomes podem variar ligeiramente entre modelos):

Controlo	O que queres	Valor habitual
<code>focus_automatic_continuous</code>	Autofoco desligado	0
<code>focus_absolute</code>	Foco fixo (só se a lente tiver motor; ver passo 4)	Medido no passo 4
<code>auto_exposure</code>	Exposição manual	1 (Manual Mode)
<code>exposure_time_absolute</code>	Exposição fixa	Medido no passo 5
<code>power_line_frequency</code>	Rede elétrica de Portugal	1 (50 Hz)
<code>white_balance_automatic</code>	Balanço de brancos fixo	0
<code>white_balance_temperature</code>	Temperatura fixa	O valor por defeito da câmara

Um controlo marcado `inactive` só tem efeito depois de desligares o automático correspondente. Se um controlo não aparecer na lista, a câmara não o tem: salta essa linha.

Cuidado: a câmara atual anuncia `focus_absolute`, mas a lente é de foco fixo e o controlo não faz nada. O passo 4 mostra como confirmar isto numa câmara nova.

Passo 3 — Posição X/Y da câmara

A posição X/Y é a posição da cabeça com o bico do T0 no centro da imagem. Como a câmara está no prato, que só se move em Z, esta posição é fixa e serve para qualquer printcore.

- 1 `G28`, `T0` e sobe o prato a uma altura segura (por exemplo `G0 Z10`).
- 2 Leva a cabeça para cima da câmara e, com o jog em passos de 0.1 mm, centra o bico na mira da Nozzle Cam.
- 3 Baixa o Z até a ponta ficar nítida a olho (ponto de partida para o passo 4).
- 4 Anota X e Y e arredonda (por exemplo `22.05 / 113.35 → 22 / 113`): o kTAMV centra o resto.
- 5 Grava em `idex.cfg`, na macro `_idex_vars`: `variable_camera_x` e `variable_camera_y`.

Esta é a única posição da câmara no sistema: a calibração automática, a manual e os fluxos completos leem todos daqui. Os scripts de afinação têm a sua própria cópia (`CAM_X` / `CAM_Y` no `ktamv_find_z.py`), que também tens de atualizar.

Passo 4 — Foco e altura Z

O foco depende de dois fatores: a lente e a distância ao bico (o Z, porque a câmara está no prato). Primeiro descobre se a lente tem motor; depois mede o Z.

4a. A lente tem foco motorizado? Só se o passo 2 mostrou `focus_absolute`. Com o bico centrado e o Z do passo 3, corre `python3 ~/ktamv_find_focus.py`.

- **A nitidez varia muito, com um pico claro:** a lente tem motor. O script deixa a câmara no melhor valor; anota-o para a linha `v4l2ctl` (passo 6).
- **A nitidez fica praticamente igual em todas as posições:** a lente é de foco fixo (é o caso da câmara atual: 53.5 a 54.8 em toda a gama). Ignora o `focus_absolute`; o foco passa a ser só o Z.

4b. Medir o melhor Z. Com a lente fixa (ou já focada em 4a):

- 1 Confirma à mão que o `Z_MIN` do script é seguro: em cima da câmara, `G0 Z<Z_MIN>` e verifica que há folga entre o bico e a câmara. Z mais baixo = prato mais perto do bico.
- 2 Ajusta no `ktamv_find_z.py` a gama (`Z_MIN`, `Z_MAX`) à volta do Z do passo 3, com cerca de 1 mm para cada lado.
- 3 `G28`, `T0`, e corre `python3 ~/ktamv_find_z.py`. Não mandes comandos pela consola enquanto corre.
- 4 O script mostra o melhor Z e a **margem** (Z com $\geq 90\%$ da nitidez e deteção total).
- 5 Escolhe o **meio da margem**, e não o pico exato: dá folga para printcores de comprimentos diferentes. Grava em `index.cfg`, `variable_camera_z`.

Se a margem chegar ao `Z_MIN` ou ao `Z_MAX`, o foco ideal está fora da gama testada: alarga-a e repete. Se o bico não for detetado em nenhum Z, a iluminação ou a exposição estão muito longe do certo: faz primeiro um ajuste à mão da exposição (passo 5, sem script) e volta aqui.

Passo 5 — Exposição

A exposição escolhe-se pelo comportamento do detetor, não pelo aspeto da imagem: uma imagem que parece boa a olho pode não ser a melhor para a deteção.

- 1 Com o bico do T0 em `camera_x` / `camera_y` e ao `camera_z` do passo 4, e os LEDs da câmara como ficam na calibração.
- 2 Ajusta a lista `EXPOSURES` do `ktamv_find_exposure.py` à gama da câmara nova (o `min/max` de `exposure_time_absolute` do passo 2).
- 3 Corre `python3 ~/ktamv_find_exposure.py`. Para cada exposição mostra deteções (tem de ser 8/8), jitter (variação do centro, em píxeis) e saturação.
- 4 Escolhe um valor **a meio da gama com 8/8**, e não o extremo, para ter margem se a luz mudar (LEDs a envelhecer, sala mais clara).

Na câmara atual, a deteção é total de 80 a 650 e o valor escolhido foi 300. Se só uma faixa estreita funcionar, o problema não é a exposição: é o detetor, e segue-se a secção **Se o detetor falhar**.

Passo 6 — Guardar tudo e ligar ao kTAMV

O que os scripts e o `v4l2-ctl` mudam na câmara perde-se quando o crowsnest reinicia. Os valores têm de ficar nos ficheiros.

`crowsnest.conf`, na `[cam 2]` — uma linha com todos os controlos, com o automático desligado antes do valor manual:

```
v4l2ctl: focus_automatic_continuous=0,auto_exposure=1,
        exposure_time_absolute=<passo 5>,power_line_frequency=1,
        white_balance_automatic=0,white_balance_temperature=<por defeito>
```

(no ficheiro, tudo numa só linha)

Se a lente tiver motor, acrescenta `focus_absolute=<passo 4a>` a seguir a `focus_automatic_continuous=0`. Reinicia o crowsnest e confirma com `v4l2-ctl -d <device>`

```
--get-ctrl=auto_exposure,exposure_time_absolute,power_line_frequency.  
idx.cfg, na _idx_vars — camera_x, camera_y (passo 3) e camera_z (passo 4).  
printer.cfg, na [ktamv]:
```

```
[ktamv]  
nozzle_cam_url: http://localhost/webcam2/snapshot?max_delay=0  
server_url: http://localhost:8085  
move_speed: 1800  
send_frame_to_cloud: false  
detection_tolerance: 1
```

Usa sempre o **snapshot** e não o **/stream**: com o stream, o servidor pode analisar um frame antigo, de antes de a cabeça parar.

Mainsail — a câmara VAOC (pré-visualização do kTAMV): URL Snapshot <http://<IP>:8085/image>, URL Stream vazio, Adaptive MJPEG, 4 FPS.

No fim: `sudo systemctl restart klipper` e, na consola, `KTAMV_SEND_SERVER_CFG`.

Passo 7 — Validar

A câmara está pronta quando os três testes abaixo passam.

- ☐ **Deteção contínua:** bico do T0 em cima da câmara, `KTAMV_START_PREVIEW` durante 30 s, depois `KTAMV_STOP_PREVIEW`. No log do servidor (<http://<IP>:8085/>), quase todas as linhas devem dizer `algorithm: 0`, com poucos milissegundos.
- ☐ **Repetibilidade:** `CALIBRATE_IDEX_XY_AUTO RECAL=1` e depois `CALIBRATE_IDEX_XY_AUTO` mais duas vezes. Os três offsets X/Y devem diferir em centésimas de milímetro.
- ☐ **Fluxo completo:** `CALIBRATE_IDEX_ALL_AUTO` do início ao fim, sem erros, e `CALIBRATE_IDEX_STATUS` com Z válido e XY final.

Se trocares também o printcore de teste por um de comprimento diferente, confirma que o T1 continua a ser detetado: é a prova de que o Z escolhido tem margem suficiente.

Se o detetor falhar

Não compenses um detetor fraco com exposições extremas: descobre em que passo ele falha, com imagens reais. Foi assim que se chegou à versão atual (v3), que funciona de 80 a 650 de exposição em vez de só 300–400.

- 1 Com o bico em cima da câmara, ao `camera_z`, corre `python3 ~/ktamv_capture_frames.py`.
- 2 Copia a pasta `~/ktamv_frames/` para o PC (WinSCP) e analisa as imagens com o detetor fora da máquina (ou envia-as para análise).
- 3 Ajusta os parâmetros abaixo em `/home/blocks/kTAMV/server/ktamv_server_dm.py`, testa com as imagens guardadas e só depois instala e reinicia o `kTAMV_server`.

O detetor funciona em duas fases: primeiro encontra a zona mais nítida da imagem (a ponta do bico), depois procura o furo escuro dentro do anel claro, só nessa zona. Os parâmetros foram afinados para 640x480 e para o tamanho com que o bico aparece nesta câmara (furo com cerca de 16 px de raio).

Parâmetro	Função	Valor	Quando mudar
<code>k (sharpness_map)</code>	Janela da medida de nitidez	15	Imagem com muito ruído: aumentar
<code>rel_thresh (focus_mask)</code>	Que parte da zona nítida entra na máscara	0.4	Máscara apanha só o reflexo: baixar

Parâmetro	Função	Valor	Quando mudar
<code>min_peak (focus_mask)</code>	Nitidez mínima para aceitar a imagem	6.0	Câmara com imagem mais suave: baixar
<code>pad (find_nozzle_hole)</code>	Folga à volta da zona nítida	30 px	Bico maior na imagem: aumentar
<code>min_r / max_r</code>	Raio aceite para o furo	5 / 30 px	Furo muito maior ou menor que 16 px
<code>max_roi</code>	Tamanho máximo da zona nítida	160 px	Bico muito maior na imagem: aumentar
Blocos do threshold adaptativo	Tamanho da vizinhança comparada	21 e 31	Cerca de 1.5–2 × o raio do furo
Circularidade mínima	Quão redondo tem de ser o furo	0.75	Furo deformado: baixar ligeiramente

Regra prática: se a câmara nova mostrar o bico com outro tamanho, escala `pad`, `min_r`, `max_r`, `max_roi` e os blocos na mesma proporção. O `ktamv_server_dm.py` original fica como `ktamv_server_dm.py_bk`, para comparação.

Valores da câmara atual

Resultado deste procedimento na primeira máquina (M-BIND), a 23/09/2026. Serve de referência para saber se uma câmara nova está a dar valores razoáveis.

Parâmetro	Valor
Dispositivo	<code>[cam 2], /dev/v4l/by-path/platform-xhci-hcd.0-usb2-0:1:1.0-video-index0</code>
Resolução / FPS	640x480 / 15
Foco	Lente fixa (<code>focus_absolute</code> sem efeito)
Posição X / Y	22 / 113
Z de foco	8.2 (pico 8.1; margem $\geq 90\%$ de 7.9 a 8.6; deteção total de 7.5 a 9.2)
Exposição	300 (deteção total de 80 a 650)
Rede elétrica	50 Hz (<code>power_line_frequency=1</code>)
White balance	Manual, 4600
Raio do furo na imagem	Cerca de 16 px
Variação do centro entre frames	Abaixo de 0.25 px